VEK 0.4 / Wekretia

Game Project Documentation

Project stage:	Early prototype / pre-production
Game direction:	Top-down 2D RPG
Engine:	Godot Engine
Programming:	GDScript
Art tools:	Krita, Aseprite
Version control:	Git and GitHub
Developer:	Daniel Abramek
Updated:	August 2026

What this document is

This is a snapshot of VEK 0.4 while it is still very early. It explains what actually works in the prototype, what I am building next, what the bigger game is supposed to become, and which ideas are still only ideas. I want the documentation to be useful without pretending that a prototype already contains an entire RPG.

Repository: github.com/Kaczelusznik/vek-0.4

Contents

1 The project in a nutshell

VEK 0.4 is a 2D RPG prototype built in Godot using GDScript. The game is set in Wekretia, a fictional world I had been building for years before I decided to turn it into a game. The setting grew through worldbuilding, stories, roleplay, characters, politics, religions, regions and a lot of smaller details that were never designed around a game engine.

That is the interesting part of the project for me: I am not inventing a world because I need a map for a game. I am trying to work out how an existing world can become something another person can actually explore.

The current prototype has a Godot project structure, a `CharacterBody2D` player, four-direction movement, Input Map actions, direction memory, directional idle/run animations, an early attack state, an early roll state, and basic menu/UI scene work. The combat is still a prototype, not a finished system.

Stabilise the player controller, clean up attack and roll behaviour, build one simple enemy interaction, and make one small location where movement, combat and interaction can be tested together instead of as isolated experiments.

LATER - What the larger game is aiming for

A story-driven top-down RPG with exploration, real-time combat, dialogue, quests, choices, consequences, different cultures and regions, and a world that feels like it existed before the player arrived.

2 Wekretia came before the game

Wekretia was not created as a game pitch. It existed long before VEK 0.4 and has been developed through personal worldbuilding and roleplay. Some events created by other people during roleplay sessions became part of the setting's canon, which means the world grew partly through authored lore and partly through interaction.

The current tone is dark fantasy with grotesque and early industrial influences. One of the important setting ideas is crystal-based technology. Crystals can be useful as resources or sources of power, but they are also dangerous. I do not want them to behave like a generic replacement for magic with a different colour.

Different regions are supposed to feel different because of their history, religion, architecture, politics and daily life, not only because one map is green and another one is snowy.

Scope matters

Wekretia is much bigger than one realistic solo game. The goal is not to simulate an entire continent. The game should show selected parts of a much larger world and make those parts feel dense enough that the rest of the setting still exists beyond the current map.

3 Why Godot and GDScript

The project was briefly explored through Java and JavaFX. That was useful as a programming exercise, but it was also a sign that I was trying to force a normal GUI framework to do the job of a game engine.

Around April 2026 I moved the project to Godot and GDScript. That gave me things the project actually needs: scenes, 2D nodes, animation tools, input mapping, physics-aware character movement and a workflow designed around games.

3.1 Current toolset

Godot Engine	Main engine, scenes, nodes, animation and game runtime
GDScript	Player logic and other gameplay/application scripts
CharacterBody2D	Current player body and movement base
AnimatedSprite2D	Directional character animation
Input Map	Named movement and action inputs
Krita / Aseprite	2D artwork and animation workflow
Git / GitHub	Version control and public development history

I am deliberately keeping the technology list small. At this stage the hard part is not adding more tools. The hard part is learning how to make the existing systems readable, predictable and fun to use.

4 What the current prototype actually does

The player controller is the main gameplay system that exists right now.

4.1 Movement and direction

The player is based on `CharacterBody2D` and moves in four directions. Input is read through named Godot Input Map actions rather than checking keyboard keys directly.

A simplified version of the movement idea looks like this:

```
direction = input_vector
velocity = direction * SPEED
move_and_slide()
```

The controller also keeps track of the last facing direction. That matters after the player stops moving because idle, attack and other directional actions still need to know which way the character is facing.

```
last_direction
```

4.2 Animation

The prototype switches between directional idle and movement animations. Side movement can reuse animation through sprite flipping, while front and back have their own visual states.

The current animation work is tied closely to gameplay state. One of the things I am learning here is that animation cannot become one giant collection of unrelated `if` statements without becoming painful to maintain.

4.3 Attack and roll states

The current code includes the first attack and roll states. The attack logic blocks normal movement for the duration of an attack and uses timing/state values so the controller knows when normal input should return. The roll follows the same general idea: it is a temporary player state with its own movement behaviour rather than normal walking with a faster speed value.

These are foundations. The game does **not** yet have a production-ready combat framework, full enemy combat, weapons, inventory or balancing.

5 Current project structure

The repository already contains more than the player script. There is early work around the main menu, pause menu, settings, save selection, UI, music, an intro and an inn scene. These are useful pieces of the project, but I still treat them as evolving prototype work rather than polished game systems.

The project is gradually moving toward a structure where scenes and scripts have clear responsibilities instead of putting every system into one file.

A simplified direction is:

```
VEK 0.4
|
|-- scenes/
| |-- player
| |-- menus
| |-- locations
| `-- ui
|
|-- scripts/
| |-- player
| |-- menus
| |-- world
| `-- systems
|
|-- assets/
|-- music/
|-- docs/
`-- project.godot
```

The exact folder names are not important. The point is to keep the project understandable when it grows beyond one player controller and a few test scenes.

6 How I want the gameplay to feel

The long-term direction is a top-down RPG, possibly with a slightly isometric presentation in some areas. The project moved away from an earlier side-view action/platformer direction because Wekretia has too much dialogue, politics, worldbuilding and regional history for me to want the game to be built mainly around platforming.

The main pillars I am aiming for are:

- exploration and atmosphere;
- real-time combat that has timing and commitment;
- dialogue and character interaction;
- quests that connect to local stories instead of existing only as checklists;
- choices with consequences that can affect people, information or local outcomes;
- environments that feel like places rather than combat rooms connected by corridors.

I like the atmosphere of games such as *Darkwood*, but VEK is not meant to copy one reference game. The goal is to build its own identity out of Wekretia rather than recreate somebody else's mechanics with different sprites.

7 Combat direction

Right now combat is mostly controller-state work. The final direction is much larger than what is implemented today.

The basic idea is that combat should have some commitment. Pressing attack should temporarily put the player into an attack state, and dodging/rolling should also have its own timing instead of being unlimited movement spam.

Things I want to explore later include:

- hitbox and hurtbox structure;
- one simple enemy with readable attacks;
- health and damage states;
- different weapon behaviour;
- armour and equipment;
- clearer combat feedback;
- environmental interactions where they actually improve gameplay.

What is not implemented

Advanced enemy AI, a finished inventory/equipment system, multiple finished weapon classes, faction combat logic and full tactical combat are future design ideas. They are not current features and should not be read as if they already exist in the repository.

8 RPG systems I want later

Once movement and combat are stable enough, the project can start behaving more like an RPG instead of a controller test.

The larger systems I currently want to prototype are:

- dialogue data and conversations;
- quests and quest state;
- NPC interaction;
- inventory and item data;
- equipment;
- factions and reputation where they serve the story;
- save/load;
- player choices and selected consequences;
- more complete UI and accessibility/settings work.

I do not want to build all of these at once. The project is already big enough because the world itself is big. The code should grow only when the previous layer is understandable enough to support the next one.

9 How I am developing it

VEK is both a game and the project I use to seriously learn game development. That changes the way I work on it.

My current loop is basically:

1. pick one small mechanic;
2. make the simplest version that actually works;
3. test it in the game instead of only reading the code;

4. debug the weird parts;
5. clean the script once I understand what it is doing;
6. only then add another layer.

This is slower than pasting a giant finished controller from a tutorial, but it leaves me with code I can explain and change later.

The main things I am learning through VEK are Godot, GDScript, gameplay programming, state-based behaviour, 2D animation, scene organisation, level design, game feel and the difference between a cool game-design idea and a system I can realistically implement.

10 Roadmap

The roadmap is intentionally small-step. It is a direction, not a promise that every item will appear on a specific date.

Four-direction movement, direction memory, idle/run animation states, attack foundation and roll foundation. Current priority: clean the controller and remove state/animation bugs before adding more combat complexity.

Create one test enemy, basic hitbox/hurtbox interaction, health/damage, readable attack timing and enough feedback to test whether the current player controls are actually fun against something that fights back.

Build a compact location with movement, one or more NPCs, interaction, a combat encounter and scene transitions. The point is to test systems together instead of keeping every mechanic in its own sandbox.

LATER - Phase 4 - RPG layer

Prototype dialogue, quests, inventory/item data, save/load and selected choice consequences. Keep the systems small until the vertical slice proves that they work together.

LATER - Phase 5 - Vertical slice

One polished small area that represents the actual game: exploration, at least one conversation, one short quest, one combat encounter, one meaningful decision, representative art/audio and a build that can be shown without opening the editor to repair it first.

11 What a finished VEK should become

The finished vision is not "the whole Wekretia wiki but playable." That would be impossible for one person and probably not fun anyway.

The goal is a focused RPG journey through selected parts of Wekretia where the world feels larger than the route the player takes. I want the player to understand places through characters, architecture, conflicts, objects and gameplay, not only through lore dumps.

From the technical side, I want a codebase where I can explain why the main systems exist and change them without being afraid that touching one animation will break five unrelated menus.

From the game-design side, I want enough systemic depth for actions to feel meaningful without building a simulation of every possible thing in the universe.

The actual goal

The point of VEK 0.4 is not to prove that I can list every RPG system in a README. The point is to slowly turn a world I have cared about for years into a game that other people can play, while getting better at programming and design with every version.

12 Current status summary

Working / prototyped	Godot project, player controller, four-direction movement, direction memory, directional idle/run animation, attack state, roll state, early UI/menu scenes
Being improved	Player state cleanup, attack/roll behaviour, animation coordination, project organisation
Next practical target	One small enemy interaction and one compact playable location
Planned RPG systems	Dialogue, quests, NPC interaction, inventory, equipment, save/load, factions/reputation, narrative consequences
Long-term direction	Story-driven top-down 2D RPG set in Wekretia

This document should change with the project. If a planned system becomes real, it moves into the current-state section. If an idea stops making sense, it can disappear instead of being kept only because it once looked good in a roadmap.